

PRACTICAL ACTIVITY 3 [CHALLENGE 3]: miniRSA Write-up

miniRSA

Hard

Cryptography

picoCTF 2019

AUTHOR: SPEEDDAY/DANNY

Hints ?

Description

123

Let's decrypt this.

Can you decrypt this [ciphertext](#)? Something seems a bit small.

5,892 users solved

87%

Liked

picoCTF{FLAG}

Submit Flag

CHALLENGE TITLE: **miniRSA**  
(<https://play.picoctf.org/practice/challenge/73?category=2&page=1&search=miniRSA>)  
PLATFORM: **picoCTF**  
AUTHOR: **speedday/Danny**  
CATEGORY: **Cryptography (Hard)**  
TOOL USED: **Kali Linux**

**Challenge Overview:**

The miniRSA challenge from picoCTF is a hard cryptography task that demonstrates a weakness in RSA when a very small public exponent is used. The challenge provides a ciphertext file along with hints suggesting that something about the encryption parameters is unusually small. Specifically, it points toward the public exponent  $e$ , asking how a small value could affect the security of the key. In RSA, the public exponent is typically chosen to balance efficiency and security, with 65537 being the common standard. However, if a very small exponent such as  $e = 3$  is used without proper padding, it can create vulnerabilities that allow attackers to recover the plaintext through mathematical manipulation instead of factoring the modulus.

**PROCESS:**

To begin solving the challenge, I downloaded the ciphertext file using the terminal in Kali Linux with the `wget` command and viewed its contents using `cat`.

```
jure@kali: ~  
Session Actions Edit View Help  
(jure@kali)-[~]  
$ wget https://challenge-files.picoctf.net/c_fickle_tempest/85842c29b0cfe5b651df70af3ae29e233ba1fb1fea0e969d0bd6328220e3d589/ciphertext  
--2026-03-13 05:10:52-- https://challenge-files.picoctf.net/c_fickle_tempest/85842c29b0cfe5b651df70af3ae29e233ba1fb1fea0e969d0bd6328220e3d589/ciphertext  
Resolving challenge-files.picoctf.net (challenge-files.picoctf.net)... 3.165.63.36, 3.165.63.56, 3.165.63.76, ...  
Connecting to challenge-files.picoctf.net (challenge-files.picoctf.net)|3.165.63.36|:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 880 [application/octet-stream]  
Saving to: 'ciphertext.1'  
  
ciphertext.1      100%[=====>]      880  --.-KB/s    in 0s  
2026-03-13 05:10:54 (28.2 MB/s) - 'ciphertext.1' saved [880/880]  
  
(jure@kali)-[~]  
$ cat ciphertext.1  
N: 29331922499794985782735976045591164936683059380558950386560160105740343201  
51336993900630753116592270894961916269862367534903043085954782570899470832180  
37053094594380993404277705800644009114318566569019827899482853099561118486869  
06152664473350940486507451771223435835260168971210087470894448460745593956840  
58653052791580254145009294657469480958488089660131751979444286297747112931978  
13131618420565017150405559640118995890028637308686795271844207890105514750678  
62907739054966183120621407246398518098981106431219207697870293412176440482900  
18355046737519023989845520117083141046048382944860347736130583874385275693868  
7673  
e: 3  
ciphertext (c): 2205316413931134031074603746928247799030155221252519872649623  
03230985833563493090582377207585470145285522183839424570842367744607103757905  
68021665396080921373182550668822136323307612446088893334417507866686778891781  
87306076425401962650981  
(jure@kali)-[~]  
$
```

The file revealed three RSA parameters: the ciphertext **c**, the modulus **n**, and the public exponent **e**. Upon inspecting these values, I observed that **e = 3**, which confirmed the hint about a small exponent. Because RSA encryption follows the formula **c = m<sup>e</sup> mod n**, using such a small exponent can result in a situation where the plaintext raised to the power of **e** does not exceed **n**, meaning the modular reduction does not occur. In this case, the ciphertext essentially becomes **c = m<sup>3</sup>**. This makes it possible to recover the original message by computing the integer cube root of the ciphertext.

To test this, I entered the values of **c**, **n**, and **e** into an RSA decoding tool, which indicated that the ciphertext could be solved using a **small public exponent attack**. By computing the cube root of the ciphertext and converting the resulting number into readable text, the plaintext message containing the flag was successfully recovered. This process demonstrates how

improper parameter choices in RSA, particularly the use of a very small exponent without padding, can significantly weaken the encryption scheme.

Kali Linux [Running] - Oracle VirtualBox

FileMachineViewInputDevicesHelp

RSA Cipher Calculator - O x

+

www.dcode.fr/rsa-cipher

OffSecKali LinuxKali ToolsKali DocsKali ForumsKali NetHunterExploit-DBGoogle Hacking DB

XZVVRT

D

C

O

D

E

E

U

X

P

L

I

Z

Search for a tool

SEARCH A TOOL ON DCODE

e.g. type 'boolean'

BROWSE THE FULL DCODE TOOLS' LIST

Results

Warning  $C' \geq N$  unexpected

Decryption using E (Small E attack)

picoCTF{n33d\_a\_lArg3r\_e\_3ed950f0}

RSA CIPHER

Cryptography > Modern Cryptography > RSA Cipher

RSA DECODER

Indicate known numbers, leave remaining cells empty.

VALUE OF THE CIPHER MESSAGE (INTEGER) C=89333441750786668677889178187306076425401962650981

PUBLIC KEY E (USUALLY E=65537) E=3

PUBLIC KEY VALUE (INTEGER) N=89333441750786668677889178187306076425401962650981

PRIVATE KEY VALUE (INTEGER) D=

FACTOR 1 (PRIME NUMBER) P=

Summary

RSA Decoder

RSA Certificate Reader

Complementary Helper too

What is the RSA cipher? (Definition)

How to encrypt using RSA cipher?

How to decrypt a RSA cipher?

How to generate RSA keys

How to recognize RSA

miniRSA

HardCryptographypicoCTF 2019

AUTHOR: SPEEDDAY/DANNY

Description

Let's decrypt this.

Can you decrypt this [ciphertext](#)? Something seems a bit small.

5,892 users solved

87% Liked

picoCTF{n33d\_a\_lArg3r\_e\_3ed950f0}

Submit Flag

Hurray! You solved this challenge.

X